

Tutorial - Semana 4 - Encontro 2

Introdução

No Encontro 1, o nível de teste passou a ter regra (`BP_GameMode`) e estado compartilhado (`BP_GameState`) próprios. Faltam dois papéis do Gameplay Framework: uma ponte entre o jogador humano e o `BP_Player` que ele controla, e um lugar que sobreviva à troca de nível para guardar dados que não pertencem a nenhum nível específico. Este encontro constrói `BP_PlayerController` e `BP_GameInstance` , e fecha a semana com o primeiro desafio de solução aberta da disciplina.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Explicar PlayerController como a ponte entre o jogador e o Pawn, e GameInstance como o objeto persistente entre níveis.
- Comparar PlayerController e GameInstance com os padrões equivalentes da Unity.
- Implementar uma variável persistente no GameInstance, funcional entre a troca de níveis.

Resultado Esperado ao Final da Semana

Um Vertical Slice operando sobre um Gameplay Framework completo: `BP_GameMode` e `BP_GameState` (Encontro 1) organizando regras e estado do nível de teste, e `BP_PlayerController` e `BP_GameInstance` (este encontro) fazendo a ponte com o jogador e garantindo persistência de um dado escolhido pelo grupo entre dois níveis distintos.

Pré-requisitos

- `BP_GameMode` e `BP_GameState` do Encontro 1 prontos e atribuídos ao nível de teste.
 - `BP_Player` funcional desde a Semana 2, controlável por Enhanced Input.
-

Antes de começar

O que você deverá possuir antes desta semana

- O projeto do Encontro 1, com `BP_GameMode` e `BP_GameState` compilados e atribuídos ao nível de teste via Project Settings.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset novo. Este encontro também trabalha exclusivamente com classes de Blueprint do próprio projeto.

Projeto esperado

O projeto do Encontro 1, mais um segundo nível mínimo (pode ser um level vazio criado apenas para testar a persistência entre níveis).

Parte 1 — PlayerController: a ponte entre jogador e Pawn

Objetivo

Compreender o PlayerController como o papel que separa a identidade do jogador do corpo que ele controla, e criar um `BP_PlayerController` customizado.

Conceito

Todo jogo com posseção de personagem precisa resolver o mesmo problema: separar quem o jogador é de o que ele controla no momento. O **PlayerController** representa a intenção e a identidade do jogador — input de alto nível, câmera, sessão — enquanto o Pawn (o `BP_Player` deste projeto) é apenas o corpo que ele possui, e que pode trocar ou perder sem que o jogador deixe de existir. Essa separação importa mesmo em um Vertical Slice de personagem único: qualquer input que não seja locomoção direta do corpo (por exemplo, abrir um inventário ou pausar o jogo, no futuro da disciplina) pertence conceitualmente ao `PlayerController`, não ao Pawn.

Passo a passo

1. Dentro de `Framework/`, criar um novo Blueprint Class herdando de "Player Controller".
2. Nomear o Blueprint como `BP_PlayerController`.
3. Abrir o `BP_PlayerController` e, no Event Graph, adicionar um Print String simples no Event BeginPlay, apenas para confirmar visualmente que essa classe está ativa durante o teste.
4. Compilar e salvar.
5. Abrir o `BP_GameMode` criado no Encontro 1 e, no painel Class Defaults, localizar o campo "Player Controller Class" e atribuir `BP_PlayerController`.
6. Testar em modo Play e confirmar, pelo Print String, que o `BP_PlayerController` customizado está ativo e que o `BP_Player` continua controlável normalmente pelo esquema de Enhanced Input da Semana 2.

Resultado esperado

Um `BP_PlayerController` customizado, atribuído ao `BP_GameMode`, ativo durante o Play, sem alterar o comportamento de locomoção já existente no `BP_Player`.

Verificando

O Print String do Event BeginPlay do `BP_PlayerController` deve aparecer na tela ao entrar em modo Play, e a movimentação do `BP_Player` deve continuar idêntica à da Semana 2 — a troca de `PlayerController` não deve, neste estágio, alterar nenhum comportamento de locomoção.

Problemas comuns

- **Esquecer de atribuir o PlayerController customizado ao GameMode:** o projeto continua usando o Player Controller padrão da engine; sempre revisar Class Defaults do `BP_GameMode`

após criar o Blueprint.

- **Adicionar lógica de locomoção diretamente no PlayerController:** locomoção pertence ao Pawn/Character (`BP_Player`); o PlayerController concentra apenas input de alto nível não relacionado ao corpo controlado.

Boas práticas

Mantenha o `BP_PlayerController` enxuto neste estágio do curso — ele ainda não tem responsabilidades além de existir corretamente como ponte; funcionalidades como abrir inventário serão adicionadas em semanas futuras, reutilizando esta mesma classe.

Comparação com Unity

O PlayerController da Unreal corresponde, em intenção, ao script de input/câmera atrelado ao GameObject do jogador na Unity — mas a Unity não separa nativamente "quem possui" de "o que é possuído": normalmente o mesmo GameObject concentra corpo e controle, e essa separação, quando existe, é uma decisão de arquitetura do time, não uma classe oferecida pela engine.

Parte 2 — GameInstance: o que sobrevive à troca de nível

Objetivo

Compreender o GameInstance como o objeto que persiste durante toda a execução do jogo, criar um `BP_GameInstance` customizado, e implementar uma variável persistente testável entre dois níveis.

Conceito

Por padrão, a maioria dos objetos de gameplay — incluindo Pawn, PlayerController, GameMode e GameState — é destruída e recriada a cada troca de nível. Toda engine estruturada precisa de algum mecanismo que sobreviva a essa transição, para guardar dados que não pertencem a nenhum nível específico: progresso, pontuação, configurações escolhidas pelo jogador. A Unreal resolve isso com o **GameInstance**: uma única instância que existe do lançamento ao fechamento do jogo, independentemente de quantos níveis forem carregados no meio do caminho.

Passo a passo

1. Dentro de `Framework/`, criar um novo Blueprint Class herdando de "Game Instance".
2. Nomear o Blueprint como `BP_GameInstance`.
3. Abrir o `BP_GameInstance` e, no painel Variables, criar uma variável simples de exemplo (por exemplo, um Integer chamado `ContadorDeTeste`), marcada como persistente durante a execução (não é necessário Save Game neste encontro — apenas sobreviver à troca de nível em memória).
4. No Event Graph, adicionar um Event BeginPlay que incrementa `ContadorDeTeste` em 1 e imprime o valor atual via Print String.
5. Ir em Edit > Project Settings > Maps & Modes (ou na seção "Game Instance Class", dependendo da versão) e atribuir `BP_GameInstance` como Game Instance Class do projeto.
6. Criar um segundo nível mínimo no projeto (File > New Level, template vazio), salvando-o com um nome claro (por exemplo, `Map_TesteGameInstance`).
7. Adicionar aos dois níveis (o nível de teste principal e o novo nível mínimo) uma forma simples de trocar de um para o outro em modo Play — por exemplo, um Trigger Volume com um nó "Open Level" apontando para o outro mapa.
8. Testar em modo Play: entrar no nível de teste principal, observar o valor de `ContadorDeTeste` impresso, trocar para o segundo nível através do Trigger Volume, e confirmar que o valor não reiniciou — apenas continuou a partir do que já estava acumulado no GameInstance.

Resultado esperado

Um `BP_GameInstance` customizado, atribuído ao projeto, com uma variável de exemplo cujo valor persiste corretamente ao trocar entre o nível de teste principal e o segundo nível mínimo criado neste encontro.

Verificando

Ao trocar de nível em modo Play, o valor impresso pelo Print String do `BP_GameInstance` não deve reiniciar do zero — ele deve continuar a contagem de onde parou antes da troca, provando que a instância realmente sobreviveu à transição.

Problemas comuns

- **Esquecer de atribuir o GameInstance customizado nas Project Settings:** o projeto continua usando o Game Instance padrão da engine, e a variável customizada nunca é inicializada; revisar Project Settings > Maps & Modes (ou a seção específica de Game Instance) após criar o Blueprint.

- **Colocar no GamelInstance um dado que deveria estar no GameState:** aplicar o critério "esse dado precisa sobreviver à troca de nível, ou só precisa ser compartilhado dentro do nível atual?" — se for o primeiro, GamelInstance; se for o segundo, GameState, já construído no Encontro 1.
- **Testar a persistência sem realmente trocar de nível:** parar e reiniciar o Play não é o mesmo teste — é necessário usar "Open Level" (ou equivalente) durante a própria sessão de Play para validar a persistência real.

Boas práticas

Ao decidir onde um dado deve morar no Gameplay Framework, sempre pergunte primeiro "isso sobrevive à troca de nível?" antes de perguntar "isso é compartilhado entre sistemas?" — a primeira pergunta já elimina GameMode e GameState como candidatos, direcionando corretamente para GamelInstance.

Comparação com Unity

O GamelInstance corresponde ao padrão `DontDestroyOnLoad` aplicado a um `GameObject` singleton na Unity (ou a um `ScriptableObject` persistente): ambos resolvem o mesmo problema de sobreviver à troca de cena/nível. A diferença arquitetural é que a Unreal oferece uma classe dedicada e única para esse papel desde a criação do projeto, enquanto a Unity depende de uma convenção manual do desenvolvedor para não recriar ou destruir esse objeto indevidamente a cada carregamento de cena.

Desafio

Cada grupo define e implementa um dado próprio que deve persistir entre níveis — pontuação, item coletado, ou estado de progresso —, com liberdade total de escolha sobre qual dado e como ele é atualizado, desde que a persistência via `BP_GameInstance` seja demonstrável na troca entre o nível de teste e o segundo nível criado neste encontro. Este é o primeiro desafio de solução aberta da disciplina, coerente com o início da transição para Studio Based Learning. O desafio não exige nenhum conteúdo além do que foi construído nesta semana, e admite soluções muito diferentes entre grupos.

Ao final da semana

Ao final da Semana 4 (Encontros 1 e 2), o Vertical Slice deve possuir um Gameplay Framework próprio completo: `BP_GameMode` e `BP_GameState` customizados organizando regras e estado do nível de teste, e `BP_PlayerController` e `BP_GameInstance` customizados fazendo a ponte com o

jogador e garantindo a persistência de um dado escolhido pelo grupo entre dois níveis. Isso corresponde ao início da linha "Framework (Módulo 2)" do roadmap descrito no PROJECT_ARCHITECTURE.md — as quatro classes `BP_GameMode`, `BP_GameState`, `BP_PlayerController` e `BP_GameInstance` agora residem na subpasta `Framework/`, substituindo as classes padrão da engine em todo o projeto.

Checklist

- ☐ `BP_GameMode` criado em `Framework/`, herdando de Game Mode Base
- ☐ `BP_GameState` criado em `Framework/`, herdando de Game State Base, vinculado ao `BP_GameMode`
- ☐ `BP_PlayerController` criado em `Framework/`, herdando de Player Controller, atribuído ao `BP_GameMode`
- ☐ `BP_GameInstance` criado em `Framework/`, herdando de Game Instance, atribuído nas Project Settings
- ☐ Segundo nível mínimo criado, com forma funcional de trocar entre os dois níveis em modo Play
- ☐ Variável persistente do desafio implementada no `BP_GameInstance` e demonstrável entre os dois níveis

Glossário

- **PlayerController:** classe nativa da Unreal que representa a identidade e o input de alto nível do jogador, separada do Pawn/Character controlado.
- **GameInstance:** classe nativa da Unreal que persiste durante toda a execução do jogo, sobrevivendo à troca de níveis.
- **Open Level:** nó de Blueprint usado para carregar um novo nível durante a execução, disparando a destruição dos objetos do nível anterior (exceto o GameInstance).
- **DontDestroyOnLoad:** padrão da Unity para impedir que um GameObject seja destruído na troca de cena, usado tipicamente para simular persistência equivalente ao GameInstance.

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Gameplay Framework in Unreal Engine. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-framework-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução ao Gameplay Framework (PlayerController, GameInstance). Disponível em: <https://dev.epicgames.com/community/unreal->

[engine/learning](#).

- UNITY TECHNOLOGIES. **Unity Manual** — GameObject, MonoBehaviour e o padrão DontDestroyOnLoad, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, vídeos introdutórios de Gameplay Framework; **Mathew Wadstein**, explicações pontuais de PlayerController/GameInstance.